Game Mechanics Document

Nuri Başar - 2021400129

1. Game World

Dungeon Escape is a third-person 3D maze game set inside a dark, atmospheric dungeon. The player is trapped deep inside a stone labyrinth and must find a golden key hidden somewhere in the maze, then bring it to the exit door to escape. The dungeon is filled with deadly spike traps and patrolling stone guards that will end the player's run on contact.

The maze is procedurally generated using a Depth-First Search recursive backtracking algorithm, producing a 12x12 grid of stone corridors with guaranteed multiple paths. The environment uses low-poly dungeon assets from the Unity Asset Store for an aesthetic view.

2. Player

The player character is a fully animated humanoid from Unity's official Starter Assets.

Controller	Unity Starter Assets ThirdPersonController
Movement	WASD keys for movement, mouse for camera
Camera	Cinemachine third-person follow camera with mouse orbit
Physics	CharacterController component (no Rigidbody)
UI Interaction	Press Escape to unlock cursor for button clicks

3. Key Object

The key is a small golden cube placed deep in the maze, guarded by a spike trap. It uses Unity's physics system:

- Rigidbody component with gravity enabled - the key is physically simulated
- The player can push the key by walking into it (handled by BasicRigidBodyPush on the player)
- The key pulses and rotates slowly to help the player locate it visually
- Tagged 'Key' so the door can identify it via OnCollisionEnter

Implementation: KeyObject.cs script manages the visual pulse. The push mechanic relies on Unity's built-in physics.

4. Door Object

The door is a tall flat cube placed at the end of one corridor path. It uses Unity's collision system as required:

- BoxCollider with Is Trigger = FALSE enables OnCollisionEnter detection
- OnCollisionEnter checks if the colliding object has the 'Key' tag

- On key contact: door animates open (90 degree rotation over 0.6 seconds via Coroutine)
- After animation: GameManager.WinGame() is called, showing the Win panel

Implementation: DoorController.cs uses OnCollisionEnter (as required by the assignment) and a Coroutine for the open animation.

5. Traps

Three spike traps are strategically placed guarding the path to the key, forcing the player to time their movement.. Each trap uses a Coroutine to alternate between safe and dangerous states:

- Safe phase (green): spikes retract underground (lasts 2.5 seconds)
- Danger phase (red): spikes rise to full height (lasts 1.5 seconds)
- Visual color feedback: material changes green/red to warn the player
- Kill detection: a second Coroutine checks player proximity every 0.1 seconds
- If the player is within 1.8 units during the danger phase, PlayerHealth.Die() is called

Implementation: TrapController.cs runs two simultaneous Coroutines — TrapCycle() for timing and ProximityKillCheck() for detection. Proximity-based detection is used because Starter Assets uses CharacterController (not Rigidbody), which does not fire OnCollisionEnter on other objects.

6. Guards

Two capsule guards patrol corridors using waypoint-based movement. Guards use two Coroutines as required:

- PatrolRoutine(): moves the guard between Waypoint A and Waypoint B continuously using Transform.MoveTowards
- ProximityCheckRoutine(): checks every 0.1 seconds if the player is within 1.8 units. if so, kills the player
- Guards face their direction of travel using Quaternion.LookRotation
- No Rigidbody on guards: Transform.MoveTowards avoids wall-sticking issues
- Move speed: 2.5 m/s with a brief 0.4 second pause at each waypoint

Implementation: GuardController.cs uses two simultaneous Coroutines for patrol and kill detection.

7. Winning & Death Conditions

Winning

- Player must locate the golden key cube in the maze
- Player pushes the key to the exit door using physics
- Key collides with door (OnCollisionEnter) → door opens → Win panel appears → Win panel shows 'You Escaped!' with Play Again and Quit buttons

Death

- Standing on a spike trap during danger phase → instant death
- Walking too close to a guard → instant death
- Falling off the maze platform → DeathZone trigger below the maze
- Death panel shows 'You Died!' with Play Again and Quit buttons

8. Background Music

Background music from the Action RPG Music Free Unity Asset Store package plays automatically on game start. The music can be toggled using the Music: ON/OFF button in the top-right corner of the HUD.

- AudioSource component with Play On Awake and Loop enabled
- MusicManager.cs handles toggle logic and updates the button label
- Press Escape to unlock cursor, then click the button to toggle, then again press Escape to take cursor back to game

9. HUD & UI

KEY: NO / YES	Top-left - A proximity trigger updates the HUD to 'KEY: YES' when the player first touches the key.
Hint Text	Bottom-center - shows contextual messages (3 seconds)
Music: ON/OFF	Top-right - toggles background music
Win Panel	Full overlay on escape - Play Again / Quit
Death Panel	Full overlay on death —-Play Again / Quit